The Event Loop Phases

10 min read Updated 24 Jul 2026 Topic 6 of 70

Why it matters

Node.js can serve thousands of simultaneous connections on a single thread, and the reason is the **event loop**. It is the scheduler at the heart of the runtime: a C-level loop, provided by the `libuv` library, that decides *which callback runs next*. Every timer that fires, every file that finishes reading, every HTTP request that lands on your server is a callback that the event loop picks up and hands to V8 to execute.

If you do not understand the phases of this loop, three categories of bug become impossible to reason about. First, **ordering bugs**: "why did my `setTimeout` callback run before my `setImmediate` callback here, but after it over there?" Second, **latency bugs**: "why does my server occasionally stall for 400ms even though every handler looks fast?" Third, **starvation bugs**: "why does this recursive callback freeze the whole process?" All three trace back to the same source of truth — the order in which the loop drains its queues.

This is also the single most reliably asked Node.js interview topic. Interviewers love it precisely because it separates people who have memorised "Node is non-blocking" from people who can predict the exact output of a snippet and explain *why*. That "why" is what this topic gives you.

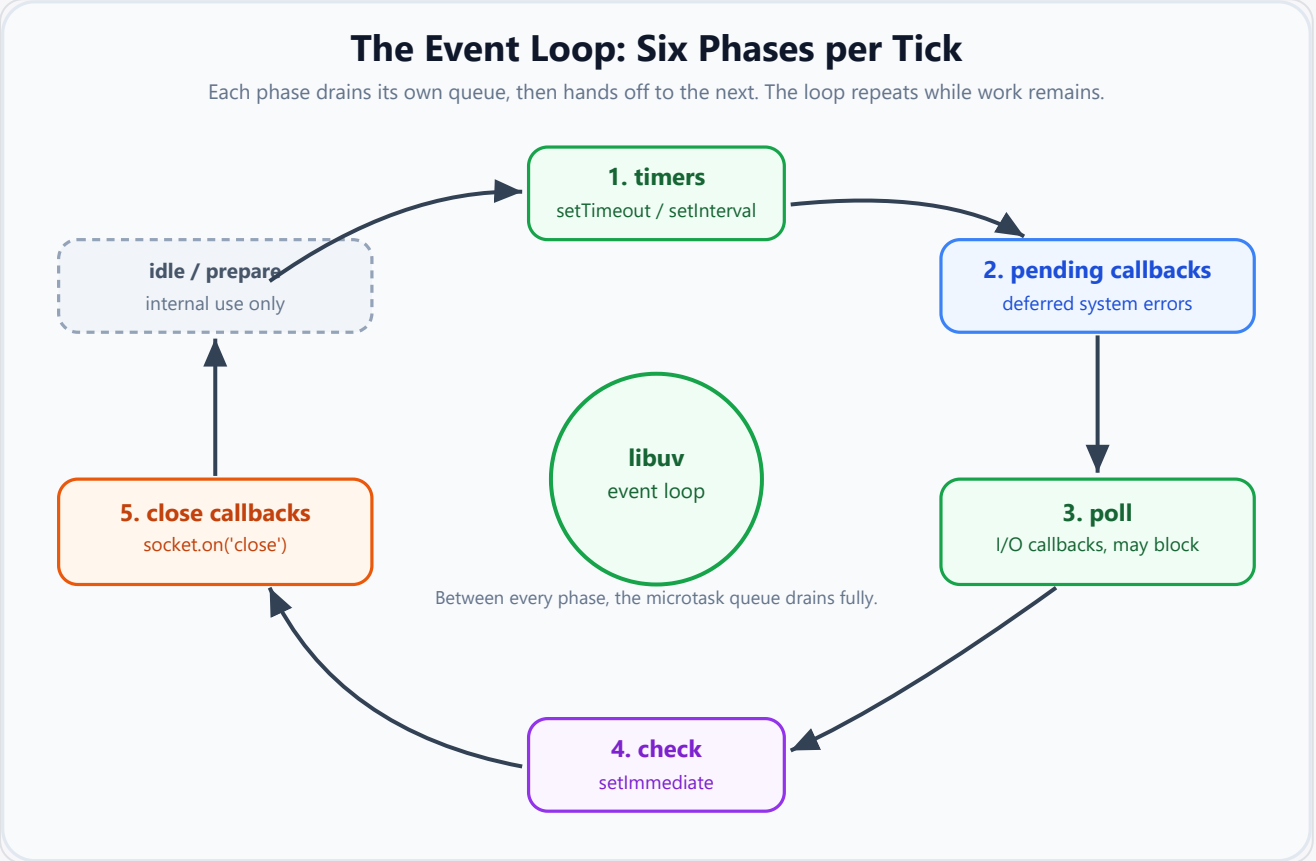
The concept, simply

Think of the event loop as a person working through a set of inboxes, in a fixed order, over and over. Each inbox — Node calls them **phases** — holds a specific kind of pending callback. The worker opens the first inbox, handles everything of that type, closes it, moves to the next inbox, and so on. When they reach the last inbox they loop back to

There are six phases, and their order never changes:

1. **timers** — runs callbacks scheduled by `setTimeout` and `setInterval` whose delay has elapsed.
2. **pending callbacks** — runs a small set of system-level callbacks deferred from a previous loop (e.g. certain TCP errors like `ECONNREFUSED`).
3. **idle, prepare** — internal to libuv only; you never schedule work here.
4. **poll** — the heart of the loop. Retrieves new I/O events and runs their callbacks (incoming data, completed file reads). This is the phase where the loop will *block and wait* if there is nothing else to do.
5. **check** — runs `setImmediate` callbacks.
6. **close callbacks** — runs `'close'` handlers, e.g. `socket.on('close', ...)`.

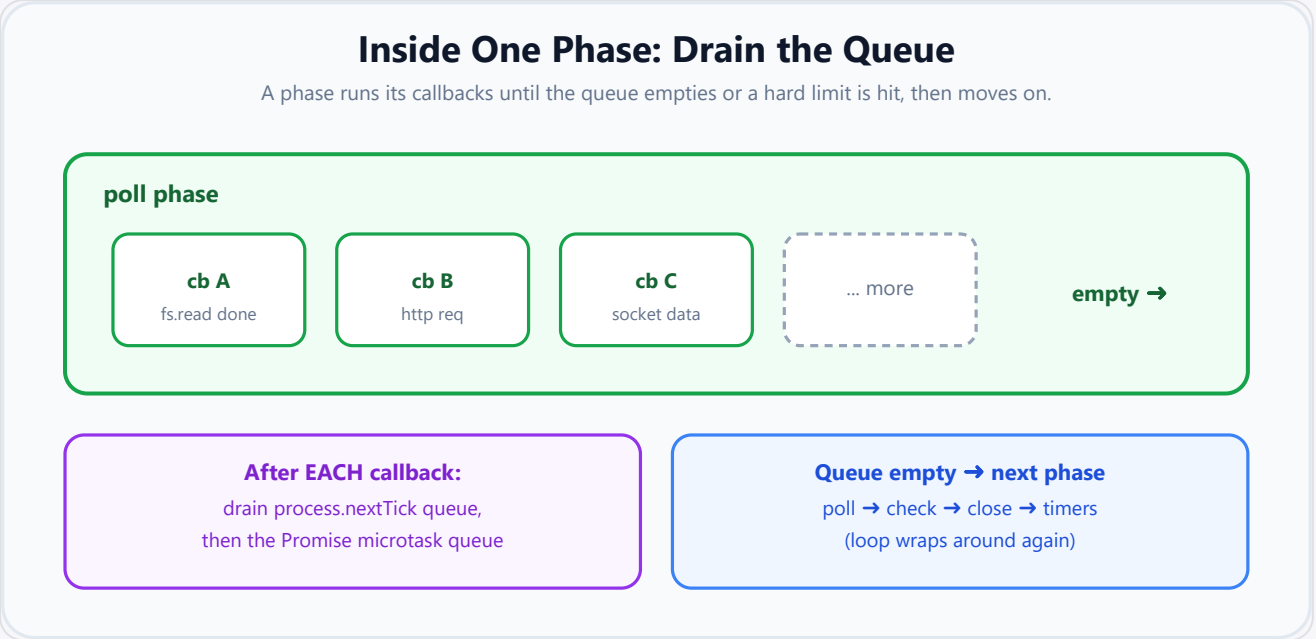
People usually shorten this to five that you actually schedule into: **timers → pending → poll → check → close**. The diagram below shows the ring the loop walks.



The event loop is a fixed ring of phases. Each phase drains its own queue before the loop advances to the next.

How it works

Each phase owns a **FIFO queue** of callbacks. When the loop enters a phase, it executes callbacks from that phase's queue *one by one* until either the queue is empty or a system-dependent maximum number of callbacks has run. Then it moves to the next phase. Callbacks you schedule while inside a phase do not necessarily run in that same phase — they land in the appropriate phase's queue and are picked up when the loop next reaches it.



Within a phase the loop drains the queue callback by callback; when it empties, control passes to the next phase.

Two subtleties do most of the interview damage:

1. The poll phase has special blocking behaviour. When the loop reaches poll, it looks at the poll queue. If there are callbacks, it runs them. If the poll queue is empty, the loop does one of two things. If there are `setImmediate` callbacks waiting, it ends poll immediately and jumps to the check phase to run them. If there is no immediate work anywhere, it will *block* in poll, waiting for the operating system to deliver an I/O event — but only until the nearest timer is due. This is how an idle Node server uses ~0% CPU: it is parked in the poll phase inside a kernel call (`epoll_wait` on Linux), asleep until something happens.

2. Between operations, two special queues drain — and they are not phases. After the currently executing callback returns (and again at each phase boundary), Node fully

between callbacks. This is why a promise `.then` always beats a `setTimeout(…, 0)`: the microtask queue is emptied before the loop is even allowed to advance to the next macrotask. We cover this fully in the microtasks topic — but you must hold the picture that phases are the coarse structure and microtasks squeeze in between every callback.

So the true rhythm of one turn is: run a phase callback → drain nextTick → drain microtasks → run the next phase callback → drain nextTick → drain microtasks → … → queue empty → advance phase. The phases give the skeleton; the microtask drains happen constantly in between.

One more framing that helps: the loop keeps running as long as there is any pending work — an open handle (a listening server, an active timer, an open socket). When no handles or requests remain, `uv_run` returns and the Node process exits naturally. That is why a bare script with a `setInterval` never exits, and why an HTTP server stays alive: the listening socket is a ref'd handle that keeps the loop turning.

Code examples

Example 1 — timers vs immediate at the top level is not guaranteed.

JAVASCRIPT Copy

```
setTimeout(() => console.log('timeout'), 0);
setImmediate(() => console.log('immediate'));

// Possible output A:        Possible output B:
//   timeout                   immediate
//   immediate                 timeout
```

At the top level the result is a race. `setTimeout(fn, 0)` is clamped to a 1ms minimum. Whether the loop reaches the timers phase and finds that 1ms already elapsed depends on how long process startup took — a few hundred microseconds of jitter decides the winner. Never rely on this ordering.

Example 2 — inside an I/O callback, the order becomes deterministic.

```
const fs = require('fs');

fs.readFile(__filename, () => {
  setTimeout(() => console.log('timeout'), 0);
  setImmediate(() => console.log('immediate'));
});

// Output (always):
//   immediate
//   timeout
```

The `readFile` callback runs in the **poll** phase. The very next phase is **check**, so the queued `setImmediate` fires first. The `setTimeout` has to wait for the loop to wrap all the way back around to the timers phase on the next iteration. This ordering is guaranteed, and it is a favourite interview question.

Example 3 — the loop stays alive because of a ref'd handle.

JAVASCRIPT Copy

```
console.log('start');
const t = setInterval(() => console.log('tick'), 1000);
// The process never exits: the interval is an active handle.
// t.unref() would let the loop exit despite the timer still existing.

// Output:
//   start
//   tick   (every second, forever)
```

Example 4 — microtasks jump the phase queue.

JAVASCRIPT Copy

```
setTimeout(() => console.log('timeout'), 0);
Promise.resolve().then(() => console.log('promise'));
console.log('sync');

// Output:
```

Synchronous code runs first. Before the loop advances to the timers phase, it drains the microtask queue, so `promise` prints before `timeout` even though the timer was scheduled first.

Common mistakes & gotchas

- **Assuming `setTimeout(fn, 0)` runs "immediately".** It runs in the next timers phase, after the current stack, after all microtasks, and after a 1ms minimum clamp — potentially much later under load.
- **Believing phase order guarantees `setTimeout` before `setImmediate`.** Only true inside an I/O callback (and there it is the *reverse*). At the top level it is a race.
- **Thinking a callback scheduled during a phase runs in that same phase.** It is queued for the next time the loop reaches the relevant phase, not appended to the phase you are currently in.
- **Forgetting the poll phase can block.** If a timer is due in 200ms and there is no other work, the loop sleeps up to 200ms in poll — that is normal, not a hang.
- **Confusing "tick" meanings.** A libuv loop *iteration* is one trip through the phases; `process.nextTick` is unrelated to that — its name is historical and misleading.
- **Expecting `setInterval(fn, 100)` to run exactly every 100ms.** It fires no sooner than 100ms after the previous run *and* only when the loop reaches the timers phase; a busy loop makes intervals drift and coalesce.

Best practices

- **Never depend on top-level timer vs immediate ordering.** If you need "run after I/O", use `setImmediate` inside the I/O callback; if you need "defer to next loop iteration", use `setTimeout(fn, 0)` and document why.
- **Use `setImmediate` to break up CPU work** so the loop can service I/O between chunks (see the blocking topic).

- Call `.unref()` on background timers (heartbeats, metrics flushers) that should not keep the process alive on their own.
- **Reason in terms of phases, not milliseconds.** "This runs in the check phase" is precise; "this runs in ~0ms" is a guess.
- **Keep individual callbacks short.** The loop is cooperative — it cannot preempt a callback that refuses to return.

⭐ Interview lens

Q: Name the phases of the event loop in order.
Timers → pending callbacks → idle/prepare (internal) → poll → check → close callbacks. In practice you schedule into timers (`setTimeout` / `setInterval`), poll (I/O), check (`setImmediate`), and close. Between every callback the nextTick and microtask queues drain.

Q: What is the output?
`setTimeout(() => console.log('A'), 0); setImmediate(() => console.log('B'));`
Indeterminate at the top level — could be A then B or B then A — because it depends on whether ~1ms has elapsed when the loop first reaches the timers phase. If you wrap both in an `fs.readFile` callback, it is always B then A, because the callback runs in poll and check comes next.

Q: Why does an idle Node server use almost no CPU?
Because the loop parks in the poll phase inside a blocking kernel call (`epoll_wait` / `kqueue` /IOCP), sleeping until the OS delivers an I/O event or the next timer is due. It is not spinning.

Q: What keeps the event loop — and the process — alive?
Pending work: active timers, open sockets, listening servers, pending thread-pool

Q: In the poll phase, when does the loop decide to move on instead of waiting?

If the poll queue is non-empty it runs those callbacks. If it is empty but `setImmediate` callbacks exist, it ends poll and goes to check. If nothing is pending anywhere, it blocks in poll until an I/O event arrives or the nearest timer's threshold is reached.

Q: Is the event loop the same as a thread?

No. The event loop is a scheduling algorithm running on the single main JS thread. libuv also manages a separate worker thread pool for certain blocking operations, but your JavaScript callbacks all execute on the one main thread, one at a time.

Cheat-card

- **Order:** timers → pending → poll → check → close, then repeat.
- **timers:** `setTimeout` / `setInterval`. **poll:** I/O callbacks (can block). **check:** `setImmediate`. **close:** `'close'` events.
- Each phase drains its FIFO queue, then advances.
- nextTick + microtasks drain *between every callback*, not as a phase.
- Inside I/O callback: `setImmediate` beats `setTimeout(0)` — always.
- Top level: timer vs immediate order is a race — never rely on it.
- Loop lives while ref'd handles exist; `unref()` releases them.

QUICK CHECK — no marks, just for you.

1. Which phase of the event loop runs `setImmediate` callbacks?

- ◯ The timers phase

- The poll phase
- The close callbacks phase

Check answer

2. Inside an `fs.readFile` callback you schedule both `setTimeout(fn, 0)` and `setImmediate`. Which runs first, and why?

- setTimeout, because 0ms elapses immediately
- They race unpredictably
- setImmediate, because the I/O callback runs in poll and the very next phase is check
- setTimeout, because timers always precede check

Check answer

3. When do the `process.nextTick` and Promise microtask queues drain relative to the phases?

- Only once at the end of every full loop iteration
- Only during the poll phase
- They form their own dedicated phase after check
- Between every callback and at each phase boundary, before the loop advances

Check answer

Previous
Running Node: Scripts, the REPL, the CLI, and nvm

Next
libuv & the Thread Pool

LEARN MORE

ARTICLES & DOCS

- The Node.js Event Loop
  nodejs.org
- Overview of Blocking vs Non-Blocking
  nodejs.org
- Don't Block the Event Loop (or the Worker Pool)
  nodejs.org

VIDEOS



- This is how the Node.js Event Loop really works
  Software Developer Diaries



- What the heck is the event loop anyway? | Philip Roberts | JSConf EU
  JSConf

Hand-picked references for this lesson.